1. Write code that creates a new paragraph element with the text "Welcome to our website!" and adds it to a div with the id "content" on the webpage. Explain what createElement and appendChild methods do

->// 1. Create a new paragraph

element const newParagraph = document.createElement("p");

// 2. Add text content to the paragraph newParagraph.textContent = "Welcome to our website!";

// 3. Find the existing div with the id "content" const contentDiv = document.getElementById("content");

// 4. Append the new paragraph to the "content" div contentDiv.appendChild(newParagraph);

2. You need to change the appearance of a button when users hover over it. Write code that finds a button with id "myButton" and changes its background color to blue and text color to white using the element.style property.

->// 1. Find the button element with id "myButton" const button = document.getElementById("myButton");

// 2. Add an event listener for when the mouse hovers over the button button.addEventListener("mouseenter", function() { // Change background color to blue and text color to white button.style.backgroundColor = "blue"; button.style.color = "white"; });

// 3. Optional: Add an event listener to reset colors when the mouse leaves button.addEventListener("mouseleave", function() { button.style.backgroundColor = ""; // Resets to original button.style.color = ""; // Resets to original });

3. Create a button that shows an alert with the message "Button was clicked!" when clicked. Also, add a second event listener to the same button that changes its text to "Clicked!" after being clicked. Explain why addEventListener is better than using onclick.

->const btn = document.getElementById("myButton");

// First listener: Shows an alert btn.addEventListener("click", function() { alert("Button was clicked!"); });

// Second listener: Changes the button text btn.addEventListener("click", function() { btn.innerText = "Clicked!"; });

4.Write a function named extractDomain that takes an email string like "john@example.com" and returns the domain part ("example.com") using only string methods. Do not use regular expressions.

->def extractDomain(email): # Splits the string into two parts at '@' and returns the second part (index 1) return email.split('@')[1]

Example usage:

print(extractDomain("john@example.com")) # Output: example.com Option 2: Using .find() and Slicing This approach manually locates the index of the @ character. python def extractDomain(email): # Find the index of the '@' symbol at_index = email.find('@')

```
# Return the part of the string starting from the character after '@'
return email[at_index + 1:]
```

Example usage:

print(extractDomain("john@example.com

5. Write a function called repeatMessage that takes a message string and a number, and returns the message repeated that many times, separated by a space. Do not use loops. Example: repeatMessage("hello", 3) should return "hello hello hello"

->function repeatMessage(message, n) { // 1. Repeat the message with a trailing space // 2. Trim the final space from the end return (message + " ").repeat(n).trim(); }

// Example usage: console.log(repeatMessage("hello", 3)); // Output: "hello hello hello" Alternative Method: Recursion If you want to avoid built-in repeat methods to show off your logic skills, you can use recursion: javascript function repeatMessage(message, n) { // Base case: if n is 1, just return the message if (n === 1) { return message; } // Recursive step: return message + space + the result of n-1 return message + " " + repeatMessage(message, n - 1); }

6. Explain what the Event Loop is in JavaScript and predict the output of the following code. Also explain why the output appears in that specific order. console.log('Start'); setTimeout(function() { console.log('Timeout 1'); }, 0); console.log('Middle'); setTimeout(function() { console.log('Timeout 2'); }, 0); console.log('End');

->. What is the Event Loop? JavaScript is single-threaded, meaning it can only do one thing at a time. The Event Loop is the mechanism that allows JavaScript to perform non-blocking operations (like fetching data or timers) by offloading those tasks to the browser (Web APIs). The process works like this: 1.Call Stack: Synchronous code is executed here one by one. Web APIs: Asynchronous tasks (like setTimeout) are sent here to wait. Callback Queue (Task Queue): Once an async task is finished, its callback function is moved to this queue. Event Loop: It constantly monitors the Call Stack. Only when the Call Stack is empty does the Event Loop take the first task from the Callback Queue and push it onto the stack to be executed. 2. Code Prediction and Output Based on the rules of the Event Loop, here Start Middle End Timeout 1 Timeout 2 3. Why does the output appear in this order? The execution follows these specific steps: Step 1: console.log('Start') is synchronous. It goes to the Call Stack, prints "Start", and is popped off. Step 2: setTimeout(..., 0) is called. Even though the delay is 0ms, it is an asynchronous Web API. The browser starts a timer and immediately moves to the next line. The callback (printing "Timeout 1") is placed in the Callback Queue. Step 3: console.log('Middle') is synchronous. It prints "Middle" immediately. Step 4: The second setTimeout(..., 0) is called. Just like the first one, its callback (printing "Timeout 2") is placed in the Callback Queue, behind "Timeout 1". Step 5: console.log('End') is synchronous. It prints "End". Step 6: Now the Call Stack is empty. The Event Loop looks at the Callback Queue and finds the "Timeout 1" callback. It pushes it to the stack, printing "Timeout 1". Step 7: The stack is empty again. The Event Loop picks up the next task, printing "Timeout.

7.Write a JavaScript function named startCountdown that logs “Time left: X” every second, starting from 5 down to 1, and finally logs “Time’s up!”. Use setInterval and clearInterval only.

->

```
function startCountdown() {
  let count = 5;

  const timer = setInterval(() => {
    if (count > 0) {
      console.log(Time left: ${count});
      count--;
    } else {
      console.log("Time's up!");
      clearInterval(timer);
    }
  }, 1000);
}

// To run the function:
startCountdown();
```

8.: Explain what Promises are in JavaScript and write code that creates a Promise to simulate checking if a user is logged in. The Promise should resolve with user data after 2 seconds if successful, or reject with an error message if the user is not found.

->What are Promises in JavaScript? A Promise is an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value. It acts as a placeholder for a value that is not necessarily known when the promise is created. Promises exist in one of three states: Pending: The initial state; the operation has not completed yet. Fulfilled (Resolved): The operation completed successfully, and the promise now has a value. Rejected: The operation failed, and the promise has a reason (error) for the failure.

```
function checkUserLogin(isLoggedIn) {
    return new Promise((resolve, reject) => {
        console.log("Checking login status...");

        // Simulate a 2-second delay using setTimeout
        setTimeout(() => {
            if (isLoggedIn) {
                // Resolve with user data if successful
                const userData = {
                    id: 101,
                    username: "JS_Developer",
                    email: "dev@example.com"
                };
                resolve(userData);
            } else {
                // Reject with an error message if the user is not found
                reject("Error: User not found or not logged in.");
            }
        }, 2000); // 2000 milliseconds = 2 seconds
    });
}

// --- How to use the Promise ---
```

```
// Case 1: Successful login
checkUserLogin(true)
    .then((data) => {
        console.log("Success:", data);
    })
    .catch((error) => {
        console.error(error);
    });
```

9.Rewrite the following Promise-based code to use async/await syntax. The code fetches user data and then fetches that user's posts. Include proper error handling. function getUserDataAndPosts(userId) { return fetchUserData(userId) .then(userData => { return fetchUserPosts(userData.id) .then(posts => { return { user: userData, posts: posts }; }); }) .catch(error => { console.error('Error:', error); throw error; }); }

->

```
async function getUserDataAndPosts(userId) {
  try {
    // Await the first call to get user data
    const userData = await fetchUserData(userId);

    // Await the second call using data from the first result
    const posts = await fetchUserPosts(userData.id);

    // Return the combined result
    return { user: userData, posts: posts };
  } catch (error) {
    // Handle any errors that occur in the try block
    console.error('Error:', error);
    throw error;
  }
}
```

10. Write code using the fetch API to get weather data from a public API. Display the weather information in a user-friendly format. Assume you have the following configuration for the API. const apiKey = 'abcd1234'; const apiUrl = `https://api.openweathermap.org/data/2.5/weather?q=Mumbai&appid=${apiK ey}&units=metric`;

->

```
[11:39, 26/03/2026] Nikita: // Function to fetch and display weather data
function getWeatherData() {
    fetch(apiUrl)
        .then(response => {
            // Check if the request was successful
            if (!response.ok) {
                throw new Error('Network response was not ok ' + response.statusText);
            }
            return response.json(); // Parse the JSON data from the response
        })
        .then(data => {
            // "data" now contains the weather information
            displayWeather(data);
        })
        .catch(error => {
            console.error('There was a problem with the fetch operation:', error);
        });
}

// Function to display the information in a user-friendly format
function displayWeather(data) {
    const { name } = data;
    const { temp, humidity } = data.main;
    const { description } = data.weather[0];
    const { speed } = data.wind;

    console.log(Weather in ${name}:);
[11:39, 26/03/2026] Nikita: console.log(Temperature: ${temp}°C);
    console.log(Condition: ${description});
    console.log(Humidity: ${humidity}%);
    console.log(Wind Speed: ${speed} m/s);
}
```

```
// Execute the function
getWeatherData();
```

Start coding or generate with AI.